Extended Abstract

SODA: Supervised Option Discovery for Dynamic Action Chunking

Motivation Action chunking policies Zhao et al. (2023); Chi et al. (2023) fix both the planning horizon T_p and execute window T_a globally across a trajectory. Real tasks are composed of subtasks with different durations: a horizon mismatched to the subtask either truncates mid-skill or pads beyond the sub-goal, making even the first executed steps suboptimal. Separately, no single execute window optimally balances reactivity and consistency across all task phases. We propose **SODA** (Supervised Option Discovery for Dynamic Action Chunking) to address both limitations jointly.

Method SODA is a hierarchical IL framework. A high-level policy π_{high} (ResNet-18 classifier) selects a discrete semantic option from the current observation. A low-level policy π_{low} generates variable-horizon action chunks conditioned on the option via a $D{+}1$ output channel; all segments are stretched to a fixed length h_{max} during training, and the predicted duration is recovered at inference by mean-pooling. A termination head β (MLP on stop-grad bottleneck features) provides reactive per-step replanning. We compare **SODA-D** (duration-channel termination) against **SODA-B** (learned β).

Implementation Push-T: circular end-effector pushes a T-block into a goal zone; $2{\times}96{\times}96$ RGB obs, 2D displacement actions, $\sim$206 demos with three VLM-labeled options ($\sim$1,430 segments). Metric: mean max block-target overlap (%) over 50 episodes, $T_{\text{max}}{=}300$. Trained on A100/H100 via Modal.

Results Performance increases monotonically with kernel size: 93.4% ($k{=}5$), 96.7% ($k{=}7$), 98.0% ($k{=}9$). A systematic study of eight β variants finds that duration termination (98.0%) outperforms every learned variant by $\geq$8 pp; the best learned variant reaches 89.7%. SODA-D $k{=}9$ outperforms DP $k{=}5$ (89.0%) with substantially reduced variance; at matched $k{=}5$, SODA-D leads by 4.3 pp. Under temporally-correlated noise, DP degrades 17.2 pp vs. only 0.3 pp for SODA-D.

Discussion Option segments averaging 17 steps (max 84) require a wider receptive field than the DP default $k{=}5$. Push-T's regular option durations leave little room for β to add value over the duration channel; more complex tasks may change this. Noise robustness is an emergent benefit of option-scoped replanning, not an explicit design goal.

Conclusion SODA demonstrates that semantic option decomposition with variable-horizon diffusion outperforms flat DP on Push-T. Duration termination is a strong and simple baseline; learned β remains an open problem. Future work targets 6-DOF tasks with VLM-supervised and LOVE-discovered options.

SODA: Supervised Option Discovery for Dynamic Action Chunking

Umar Padela
Department of Aeronautics & Astronautics
Stanford University
umarp@stanford.edu

Neetish Sharma
Department of Computer Science
Stanford University
neetishs@stanford.edu

Abstract

Action chunking policies fix the planning horizon and execute window globally across a trajectory, misscoping subtasks of variable duration and leaving the consistency–reactivity trade-off unresolved. We propose SODA (Supervised Option Discovery for Dynamic Action Chunking), a hierarchical imitation learning framework in which a high-level classifier π_{high} selects discrete semantic options and a low-level diffusion policy π_{low} generates variable-horizon action chunks conditioned on the active option. A temporal stretching mechanism trains π_{low} on segments of heterogeneous length by linearly interpolating each to a fixed maximum horizon while supervising a duration output channel with the normalized native length; the predicted duration is recovered at inference by mean-pooling. We additionally train a termination head β and systematically evaluate eight variants across four design axes. On Push-T, SODA-D (k=9) achieves 98.0% mean max block-target overlap versus 89.0% for the Columbia Diffusion Policy baseline, with substantially reduced variance. A key negative result: the simple duration channel outperforms every learned β variant by at least 8 percentage points. Under temporally-correlated action noise, DP degrades by 17.2 pp while SODA-D degrades by only 0.3 pp, an emergent benefit of option-scoped replanning.

1 Introduction

Action chunking has emerged as a foundational paradigm for imitation learning in robot manipulation (Zhao et al., 2023; Chi et al., 2023). Rather than predicting a single action, these policies forecast a sequence of T_p future steps and execute the first $T_a \leq T_p$ before replanning a structure that captures temporal correlations across actions and handles the multi-modal ambiguity inherent in human demonstrations. The resulting policies have achieved impressive performance on complex, contact-rich tasks that confound single-step approaches.

Yet the fixed temporal windows that make action chunking tractable also constrain how well these policies adapt to tasks composed of subtasks with varying durations. Consider pushing a T-shaped block into a goal orientation: the policy must first reposition the end-effector near the block, then translate it across the workspace, then pivot it into the target pose three phases with different characteristic lengths. A policy planning over a fixed horizon T_p must either truncate a coherent plan mid-subtask, when the subtask runs long, or pad the plan with spurious actions beyond the sub-goal, when it runs short. In both cases the model generates predictions conditioned on the wrong temporal context. Crucially, this misscoping propagates directly into the actions that are actually executed: even the first T_a steps that reach the robot are drawn from a horizon-mismatched plan, making them suboptimal relative to a plan scoped to the true subtask duration.

A separate tension arises in choosing how many of those steps to execute before replanning (Liu et al., 2024). Replanning frequently keeps the policy responsive to disturbances, but successive

chunks often contradict each other slightly, producing jittery motion. Replanning infrequently yields smooth, consistent execution, but delays the policy's reaction to unexpected state changes. This consistency–reactivity tradeoff has no satisfactory resolution under a fixed T_a, and the optimal balance shifts across task phases in ways a single global value cannot accommodate.

In this work, we propose **SODA** (Supervised Option Discovery for Dynamic Action Chunking), a hierarchical imitation learning framework that addresses both of these issues. SODA trains a high-level policy π_{high} to select discrete semantic *options* (Sutton et al., 1999) one per subtask phase and a low-level policy π_{low} that generates action chunks conditioned on the current option. Because option boundaries are aligned with natural task transitions, π_{low} plans over a horizon matched to the actual subtask duration rather than a global fixed window. For the consistency–reactivity tension, we train a termination head β alongside π_{low} that monitors execution at every step and signals when to replan. We compare this learned mechanism against a simpler baseline, **SODA-D**, that infers option completion from a dedicated duration output channel without any reactive early exit. Both strategies are described in detail in Section 3.

Central to SODA is a learned decomposition of the task into discrete semantic options, each corresponding to a distinct phase of manipulation. This option structure determines the boundaries at which π_{low} is conditioned and retrained, enabling the variable-horizon planning that the architecture depends on. How options are discovered and what quality of decomposition is required for the framework to work well are questions we explore in this paper.

This paper makes four contributions. First, we present a hierarchical IL architecture combining option-conditioned diffusion with a learned duration channel for variable-horizon planning and two execution strategies for the consistency–reactivity trade-off. Second, we conduct a systematic termination study spanning eight variants of β, varying feature source, teacher distribution, training signal, and gradient flow. Third, we show that semantic decomposition outperforms flat Diffusion Policy on Push-T (SODA-D k=9: 98.0% vs. DP k=5: 89.0%), with hierarchy contributing even at matched kernel size (SODA-D k=5: 93.4% vs. DP k=5: 89.0%, +4.3 pp). Fourth, we report a negative result: the duration channel outperforms all learned β variants by at least 8 percentage points, establishing duration termination as a strong baseline for option switching in this task.

2 Related Work

Action chunking and generative policies. ACT (Zhao et al., 2023) and Diffusion Policy (Chi et al., 2023) demonstrate that predicting multi-step action sequences substantially outperforms single-step policies on dexterous manipulation tasks, both by capturing temporal correlations across steps and by representing the multi-modal structure of demonstration data through expressive generative models. These methods fix the planning horizon T_p and execute window T_a uniformly across the full trajectory, without regard to the variable durations of individual subtasks. SODA extends the Diffusion Policy U-Net architecture to support option-conditioned variable-horizon planning, directly addressing the structural limitations that arise from these fixed temporal windows.

Bidirectional Decoding. Liu et al. (2024) identify the consistency–reactivity trade-off as a central limitation of fixed-T_a execution and resolve it through Bidirectional Decoding (BID): at each step, N candidate chunks are generated and scored by backward coherence to recently executed actions, enforcing consistency, and by forward contrast, promoting diversity, with the highest-scoring candidate selected for execution. Setting T_a=1 makes this equivalent to per-step replanning without jitter. Notably, BID addresses the consistency–reactivity tension entirely at test time and requires no changes to the training procedure, but it leaves the planning horizon fixed chunks carry no semantic scope over the subtask they cover. SODA instead addresses this tension through a learned termination head β trained alongside π_{low}, which provides reactive replanning at the cost of a single MLP forward pass rather than N full diffusion passes.

Options framework. Sutton et al. (1999) extend the Markov decision process formalism to Semi-MDPs through the options framework, defining a temporally-extended action as a triple of a sub-policy, an initiation set, and a stochastic termination condition β. This framework provides the theoretical foundation for SODA's two-level hierarchy. SODA instantiates it in an offline imitation learning setting, supervising the termination condition from demonstration segment boundaries rather than reward signals.

Variable-horizon planning. Pfrommer et al. (2024) augment flow-matching policies with a horizon channel, training via reinforcement learning to minimize task completion time and thereby adapt prediction length to the current state. SODA borrows the duration-channel mechanism but operates entirely in a supervised IL regime: option boundaries derived from demonstrations supply the horizon supervision, removing the need for environment rewards or online interaction.

Skill discovery. CompILE (Kipf et al., 2019) learns to segment demonstrations into variable-length latent skills through a differentiable probabilistic segmentation model trained to minimize reconstruction loss over full trajectories. LOVE (Jiang et al., 2022) frames option discovery as a compression problem, minimizing the description length of the inferred skill sequence to prevent degenerate solutions in which a single trivial skill spans the entire trajectory. While both methods discover skills without manual annotation, they optimize reconstruction quality rather than producing boundaries aligned with task-relevant subtask transitions. SODA takes a complementary approach: boundaries are specified through VLM supervision, trading discovery generality for semantic precision at the subtask level.

Weakly-supervised task decomposition. TACO (Shiarlis et al., 2018) supervises modular policies using task sketches ordered sub-task label sequences without temporal boundary annotations recovering segmentations through dynamic time warping between demonstrations and sketch templates. Policy Sketches (Andreas et al., 2017) condition modular RL policies on analogous named sub-task sequences over a shared skill library. Both relax the annotation burden relative to frame-level labels, but operate in settings with discrete task programs and do not produce variable-horizon continuous action chunks conditioned on a generative diffusion model.

Deep Skill Chaining. Bagaria and Konidaris (2020) discover options in continuous high-dimensional state spaces by iteratively growing initiation sets backward from a sparse goal-reward signal through online environment interaction. The approach requires an RL training loop and cannot be applied to the offline imitation learning setting that SODA targets.

Gap. Taken together, prior work addresses the two structural limitations of action chunking in isolation and predominantly through reinforcement learning. BID resolves the consistency–reactivity trade-off at test time but leaves the planning horizon fixed. Variable-horizon methods resolve the planning-horizon mismatch through RL objectives. Skill discovery methods operate in the imitation learning setting but optimize reconstruction rather than control-relevant boundaries. SODA is, to our knowledge, the first framework to address both limitations jointly within a pure IL setting: semantic options scope the planning horizon to the current subtask, and a learned termination head β provides reactive replanning on demand. SODA-D, which combines dynamic horizons with a simpler duration-based termination rule, serves as the primary baseline for evaluating the contribution of learned termination.

3 Method

3.1 Option Discovery

For Push-T, we define three options corresponding to the distinct phases of the task: *reposition* (the agent moves toward the block without making contact), *linear-push* (the agent drives the block in translation), and *pivot-push* (the agent rotates the block into the target orientation). Segment boundaries are determined using a VLM, which is prompted with demonstration frames to label the active phase at each timestep. Across $\sim$206 demonstrations, this yields $\sim$1,430 option segments with a mean length of $\sim$17 steps. Figure 1 visualizes representative segments for each option.

3.2 Low-Level Policy π_{low}

π_{low} extends the Columbia Diffusion Policy (Chi et al., 2023) 1D U-Net hybrid image policy. The observation encoder is a ResNet-18 with GroupNorm and spatial softmax pooling. The U-Net backbone operates over the action sequence with `down_dims`$= [512, 1024, 2048]$ and a convolutional kernel size k swept over $\{5, 7, 9\}$, where k controls the temporal receptive field of the denoising backbone.

Option conditioning is introduced via a learnable embedding Embed(ω, d=32) concatenated to the global FiLM conditioning vector before being injected into each U-Net residual block. The action

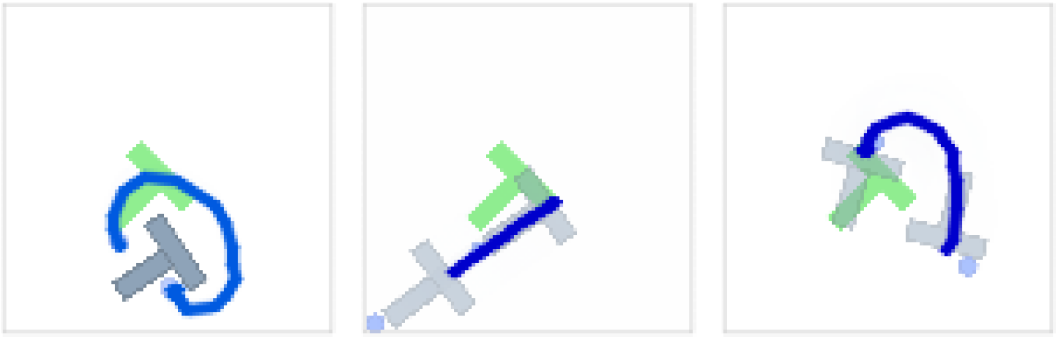

Figure 1: The three VLM-labeled options for Push-T used by SODA-D, shown on representative segments. **Reposition**: agent approaches the block without contact. **Linear-push**: agent drives the block in translation. **Pivot-push**: agent rotates the block into the target orientation.

output is extended from D to $D{+}1$ dimensions, where the extra channel encodes the predicted duration of the current option segment.

The key training mechanism is temporal stretching. Each option segment has a native length h that varies across demonstrations and options. To produce uniform-length training targets, the h actions in each segment are linearly interpolated to h_{max} steps, where h_{max} is set to the length of the longest segment in the dataset ($\sim$84 steps for Push-T). All segments are therefore stretched to the same length regardless of their original duration, with the relative timing of actions preserved. The $D{+}1$-th channel of each stretched target is set to the normalized native length h/h_{max}, constant across all h_{max} positions. The model thus learns to embed the original segment duration into every step of its output.

At inference, π_{low} generates a full h_{max}-length action chunk. The native duration of the predicted option segment is recovered by mean-pooling the $D{+}1$-th channel over the chunk and scaling back: $\hat{h} = \text{round}(\bar{d} \times h_{\text{max}})$. The first $\hat{h}$ actions are then executed (subject to the T_a-step replan cadence), after which π_{high} is queried for the next option.

For training, one sample is drawn per option segment per epoch, selecting a random anchor frame within the segment as the conditioning observation.

π_{low} is trained with DDPM ε-prediction over 100 diffusion steps using AdamW (lr=10^{-4}, wd=10^{-6}, batch size 64) with a cosine learning rate schedule and a 5-epoch linear warmup, for 500 epochs total.

3.3 High-Level Policy π_{high}

π_{high} is a ResNet-18 initialized from ImageNet weights, with a small MLP head that maps the encoded observation to a distribution over options. It is trained in parallel with π_{low} as a supervised classifier: given any frame from the demonstration dataset, it predicts which option the expert is executing at that timestep. Training uses all frames across all demonstrations for 100 epochs, and the best checkpoint is selected by classification accuracy on held-out validation episodes, reaching $\sim$92% on Push-T.

3.4 Termination Head β

The termination head is an MLP with two hidden layers of dimension 256, producing a single scalar logit from either stop-gradient pooled U-Net bottleneck features or ResNet-encoded observations. It is trained with binary cross-entropy, where the positive class labels the final frame of each option segment indicating that the current option is complete.

The design of β involves four independent choices that we evaluate systematically in the termination study (Section 5.1). The first is *feature source*: whether β conditions on U-Net bottleneck features, extracted at a fixed DDIM timestep during the denoising process, or on raw ResNet observation encodings. The second is *teacher actions*: when using bottleneck features, whether they are extracted

using expert actions or DDIM-generated actions. The expert variant conditions the U-Net on ground-truth demonstration actions to produce bottleneck activations, but this creates a training-inference distribution gap at inference time, β must operate on bottleneck features conditioned on the model's own generated actions, not expert actions it does not have access to. The DDIM variant closes this gap by running a short DDIM chain to generate actions before extracting bottleneck features, so the distribution of activations seen during β training matches what is seen at inference. The cost is additional compute per training step, since each sample requires a partial diffusion forward pass. The third is *training labels*: completion-only boundaries versus completion-plus-escape labels, where escape relabeling randomly substitutes an incorrect option at $p{=}0.5$ and sets $\beta{=}1$, training the head to additionally detect when the policy is executing under the wrong option. The fourth is *gradient flow*: whether the observation or bottleneck encoder is frozen during β training or updated jointly with BCE loss weighted by $\lambda{=}0.01$.

At inference, SODA-B evaluates β at every step and checks two thresholds. If the logit exceeds θ_{high}, control returns to π_{high} to select a new option. If it exceeds θ_{low}, control returns to π_{low} to replan the current chunk without changing the option. Otherwise, the next action in the current chunk is executed, with a full replan triggered every T_a steps.

3.5 Execution Strategies

At the start of each episode, π_{high} selects an initial option ω from the current observation, and π_{low} generates a full h_{max}-length action chunk conditioned on (s, ω). We compare two strategies for managing replanning and option switching during execution.

SODA-D executes chunk actions sequentially and replans the chunk every $T_a{=}8$ steps by calling π_{low} with the current observation and option. Option switching is driven entirely by the duration prediction: when the number of steps executed within the current option reaches $\hat{h}$, π_{high} is queried to select a new option. There is no reactive early exit between scheduled replan steps.

SODA-B augments SODA-D with per-step termination monitoring via β. At every step, if β signals option completion, π_{high} is called immediately; if it signals that the current chunk should be replanned without changing the option, π_{low} is called immediately. The T_a-step replanning cadence is preserved as a fallback when β does not fire, ensuring baseline execution consistency.

4 Experimental Setup

We evaluate SODA on Push-T (Chi et al., 2023), a 2D manipulation task in which a circular end-effector must push a T-shaped block into a goal zone. The policy observes a stack of two consecutive $96{\times}96$ RGB frames and outputs 2D end-effector displacement actions. We train on $\sim$206 expert human demonstrations from the Columbia dataset, annotated with the three VLM-labeled options described in Section 3.

Our baseline is the official Columbia Diffusion Policy checkpoint trained on the same dataset for 3050 epochs with kernel size $k{=}5$. We train SODA models from scratch under the same data and evaluation conditions. All experiments are run on A100/H100 GPUs via Modal.

Performance is measured as the mean maximum block-target overlap achieved within $T_{\text{max}}{=}300$ environment steps, expressed as a percentage and averaged over 50 episodes with fixed seeds. This metric rewards both task completion and partial progress. Unless a study explicitly varies it, all SODA models are evaluated with SODA-D ($T_a{=}8$, duration termination). Experiments are organized as a sequence of ablation studies, each fixing one design decision before proceeding to the next: kernel size, termination mechanism, receding horizon window, and finally a direct comparison against the DP baseline.

5 Results

5.1 Quantitative Evaluation

5.1.1 Kernel Size Study

We first sweep the convolutional kernel size $k \in \{5, 7, 9\}$ of the diffusion U-Net, training each variant for 500 epochs under identical conditions (duration termination, T_a=8) and evaluating every 50 epochs. Performance increases monotonically with k: the best checkpoints achieve 93.4% at k=5, 96.7% at k=7, and 98.0% $\pm$ 13.0% at k=9 (Figure 2). We attribute this trend to the structure of option segments: with a mean length of $\sim$17 steps and a maximum of 84 steps, the U-Net must capture dependencies across substantially wider temporal windows than the k=5 default used by flat Diffusion Policy. Longer option segments in particular may require the larger receptive field to plan coherently across their full duration, and a narrow kernel that suffices for flat action chunks is insufficient for the stretched, option-conditioned targets that π_{low} must generate. All downstream experiments use the k=9 checkpoint at epoch 450.

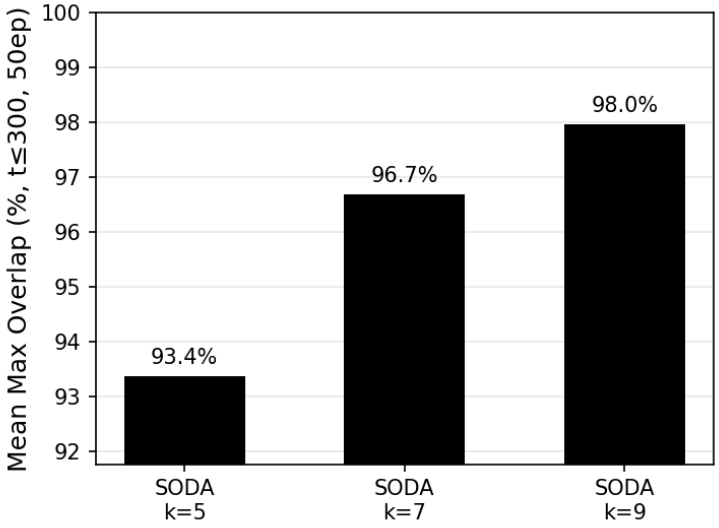

Figure 2: Kernel size study: best-epoch mean max overlap (%) over 50 episodes for SODA $k \in \{5, 7, 9\}$, all with duration termination and T_a=8. Performance increases monotonically with k; $k = 9$ (98.0%) is selected for all downstream experiments.

5.1.2 Termination Study

We systematically evaluate eight variants of SODA-B across the four design axes described in Section 3, using the k=9 backbone. Full results are shown in Table 1.

Among frozen variants, using expert actions to extract bottleneck features outperforms DDIM-generated actions (89.0% vs. 84.1%), suggesting that the distribution gap between expert and diffusion-sampled trajectories hurts completion detection. Two factors may limit the DDIM result. First, DDIM training was expensive roughly 3–4$\times$ slower per step and was therefore limited to 100 epochs; it is possible that performance would continue to improve with longer training. Second, inference uses DDPM over 100 denoising steps, while training uses a short DDIM chain; the two samplers do not produce identical trajectory distributions, which may leave a residual gap in the bottleneck features β must interpret. Running full DDPM during training would provide a tighter match, but the compute cost is prohibitive and is left for future work. Escape relabeling consistently degrades performance: observation-based training with boundary-only labels achieves 86.0%, while adding escape labels drops it to 74.9%, indicating that the relabeled negatives introduce more noise than useful signal on Push-T. Joint training helps in some configurations the DDIM bottleneck

variant improves from 84.1% to 89.7% when trained jointly but degrades in others, and no variant approaches the duration termination baseline.

The duration baseline achieves 98.0%, outperforming every learned β variant by at least 8 percentage points. This is a strong negative result: despite systematic exploration of the design space, a simple scalar readout of the duration channel is a more reliable termination signal than any learned head on this task. Duration termination is adopted for all subsequent experiments.

Table 1: SODA-B termination study. Mean max overlap (%) at best checkpoint, 50 episodes, $k = 9$, T_a=8. Frozen = stop-grad bottleneck; Joint = λ=0.01, gradients flow into backbone. Duration termination (SODA-D, 98.0%) is shown for reference; all SODA-B variants trail by $\geq$8 pp.

	Duration termination	Bottleneck expert completion	Bottleneck DDIM completion	Bottleneck DDIM completion + escape	Observation completion	Observation completion + escape
Frozen β (stop-grad)	98.0	89.0	84.1	78.3	86.0	74.9
Joint β (stage 3)	—	87.3	89.6	73.5	84.2	71.7

5.1.3 Receding Horizon Study

We sweep the execute window $T_a \in \{2, 4, 6, 8, 10, 16, \text{open-loop}\}$ on the fixed k=9 checkpoint, without any additional training. Performance peaks at T_a=8 (98.0%), with T_a=6 close behind at 96.0%. Larger windows degrade noticeably T_a=16 drops to 87.7% and open-loop execution, which commits to the full predicted chunk without replanning, collapses to 81.7% (Figure 3). The results confirm that periodic within-option replanning is important for task success: the temporal stretching mechanism produces coherent suffix plans, but stale actions accumulate error over long execute windows. T_a=8 is used for the final comparison.

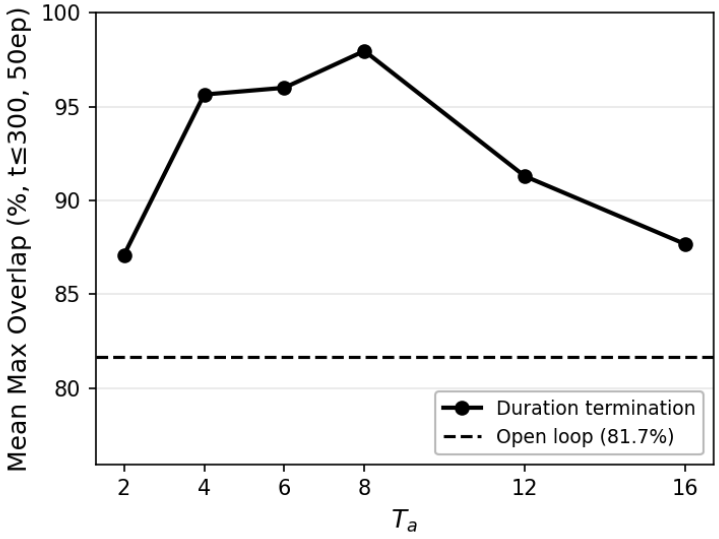

Figure 3: Receding horizon study: mean max overlap (%) vs. execute window T_a, applied to the best SODA-D ($k = 9$) checkpoint without retraining. T_a=8 is optimal; the dashed line shows the open-loop baseline (81.7%), which executes the full predicted chunk without any replanning.

5.1.4 SODA vs. Diffusion Policy

Table 2 compares SODA-D against the Columbia Diffusion Policy baseline. SODA-D at k=9 achieves 98.0% $\pm$ 13.0%, outperforming DP at k=5 (89.0% $\pm$ 22.6%) by 8.9 percentage points. Beyond the mean, SODA-D reduces variance substantially, suggesting that semantic decomposition suppresses the long failure tail present in flat DP rollouts.

Because SODA-D at k=9 differs from the DP baseline in both architecture (hierarchical vs. flat) and receptive field (k=9 vs. k=5), we also compare at matched kernel size. SODA-D at k=5 achieves 93.4% vs. DP's 89.0%, a +4.3 pp improvement. The direction is consistent and attributable to the hierarchical decomposition rather than receptive field. Figure 4 further shows that SODA-D completes the task faster across all time budgets from T=125 to T=300, reflecting the benefit of option-scoped planning.

Table 2: Main comparison: mean max block-target overlap (%, $t \leq 300$, 50 episodes). Each entry reports a single training run evaluated over 50 fixed-seed episodes.

Method	Mean (%)
DP $k = 5$ (Columbia baseline)	89.0
SODA-D $k = 5$	93.4
SODA-D $k = 9$	**98.0**

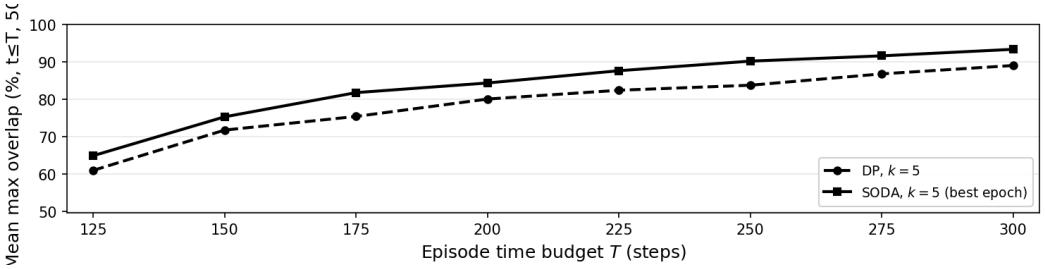

Figure 4: Mean max overlap (%) vs. episode time budget T, for SODA-D $k = 5$ and DP $k = 5$ at matched kernel (50 episodes each). SODA-D leads DP at every time budget from $T = 125$ to $T = 300$, demonstrating faster task completion in addition to higher final overlap.

5.1.5 Noise Robustness

We evaluate robustness to temporally-correlated action noise using a velocity-proportional AR(1) noise model. At each execution step, the intended displacement action a_t and the actual agent position from the observation are used to compute the velocity $v_t = a_t - p_{t-1}$. A noise signal is then generated as an AR(1) process over this velocity: $z_t = \rho \cdot z_{t-1} + (1 - \rho) \cdot (s_t \odot v_t)$, where s_t is a random sign-and-magnitude seed sampled uniformly from $[\pm 0.5, \pm 1.5]$ at each step and ρ=0.9 controls temporal correlation. The perturbed action is $\tilde{a}_t = a_t + \eta \cdot z_t$, so η=0 is identity and η=1 corresponds to approximately 100% velocity-magnitude noise sustained across time. Both models are evaluated at k=5 with T_a=1 replanning at every step to control for receptive field and isolate the effect of noise.

As shown in Table 3, DP degrades sharply under noise, dropping 17.2 percentage points from 85.2% to 68.0%. SODA-D is substantially more robust, falling only 0.3 percentage points from 87.2% to 86.9%. We attribute this robustness to the combination of per-step replanning (T_a=1) and option-scoped planning horizons: because π_{low} replans every step over a horizon matched to the current subtask, each new plan is anchored to the current observed state and scoped to a semantically meaningful objective, limiting the accumulation of noise-induced drift. Note that T_a=1 also means neither policy benefits from chunk consistency both are fully reactive. This exposes SODA-D to potential jitter from constant replanning at η=0, and applying BID (Liu et al., 2024) on top of SODA-D could further improve clean performance by restoring chunk-to-chunk consistency without sacrificing the option-scoped planning horizon that confers noise robustness.

Table 3: Noise robustness study. Mean max overlap (%) under velocity-proportional AR(1) noise at η=0 (no noise) and η=1 (full noise), both at k=5, T_a=1.

Method	η=0	η=1
DP	85.2	68.0
SODA-D	**87.2**	**86.9**

5.2 Qualitative Analysis

Figure 1 shows representative segments for each of the three heuristic options. The spatial extent and directionality of each option are visually distinct: reposition segments show the end-effector moving toward the block along varying approach paths; linear-push segments show sustained translational contact driving the block across the workspace; pivot-push segments show tight rotational motion aligning the block with the goal orientation. The visual separation confirms that the contact-based boundary rule produces semantically coherent segments, and that the option labels carry meaningful structure for π_{low} to condition on.

Figure 5 illustrates a representative failure mode from the SODA-B termination study (bottleneck_expert, frozen variant). At step 75 of the rollout, the agent is executing the PIVOT-PUSH option. The termination head outputs β=0.48, well below the firing threshold of 0.9, so no option transition is triggered. Simultaneously, the low-level policy predicts a native duration of $\hat{h}$=1 step the minimum possible meaning π_{low} has effectively decided the current option is complete and is issuing a single, near-zero action. Because β does not fire, π_{high} is never queried for a new option, and the policy enters a degenerate loop: it replans each step, continues to predict $\hat{h}$=1, and executes a near-stationary action, leaving the robot frozen in place for the remainder of the episode. This failure exposes a key coupling issue in SODA-B: the duration channel and the termination head can disagree, and when β is too conservative, the low policy's internal belief that the option is over cannot propagate into an actual option switch.

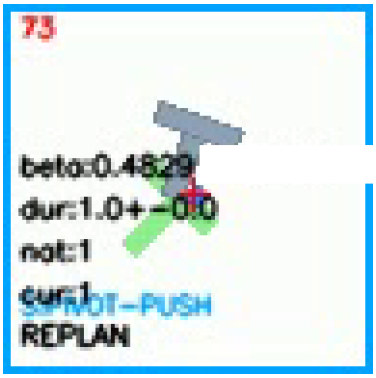

Figure 5: Failure case from the SODA-B termination study (bottleneck_expert, frozen variant), step 75. The termination head outputs β=0.48 (threshold 0.9), so no option switch is triggered. The low policy simultaneously predicts $\hat{h}$=1 step (nat:1), indicating it believes the PIVOT-PUSH option is complete. With β not firing, π_{high} is never called, and the robot executes a near-zero action every step for the remainder of the episode.

Figure 6 shows a second failure mode from SODA-D (k=9, epoch 450). At step 153, the block has been pushed into the corner of the workspace, leaving the end-effector stranded with no clear path to re-engage the block. The policy continues to execute the REPOSITION option but makes no meaningful progress. This is a classic limitation of imitation learning: the expert demonstrations cover the nominal task trajectory, but recovery from states reached through accumulated error is not represented in the training data. Once the agent perturbs itself into such an out-of-distribution configuration, it has no behavioral template to draw on and tends to persist in or deepen the failure. A natural remedy is DAgger, which would query the expert in precisely these off-nominal states to extend coverage and train a policy capable of recovery.

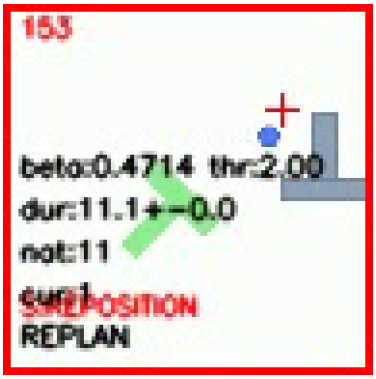

Figure 6: Out-of-distribution failure case from SODA-D (k=9, epoch 450), step 153. The block has been pushed into the corner of the workspace, a configuration not present in the expert demonstrations. The policy remains in the REPOSITION option but cannot re-engage the block, illustrating the standard IL limitation: without expert coverage of recovery states, the policy has no corrective behavior to draw on. DAgger-style interactive imitation learning would address this by soliciting expert corrections from off-nominal states.

Figure 7 shows a failure mode rooted in π_{high} misclassification. At this state, the block is partially rotated and the end-effector is in contact near the top a geometrically ambiguous configuration where the expert would begin a reposition, but π_{high} predicts PUSH with 0.94 confidence. Because the option boundary between reposition and push is determined by subtle contact geometry that may not be cleanly separable from image observations alone, π_{high} learns a decision boundary that misfires on these edge cases. The downstream consequence at inference is a disagreement loop: π_{high} continuously selects the PUSH option, while π_{low} which has learned that push segments end when the block reaches a certain configuration predicts a short remaining duration and outputs near-zero actions. Neither policy can break the deadlock: π_{high} is committed to PUSH, and π_{low} effectively believes the push is already complete. The robot stalls until the episode terminates.

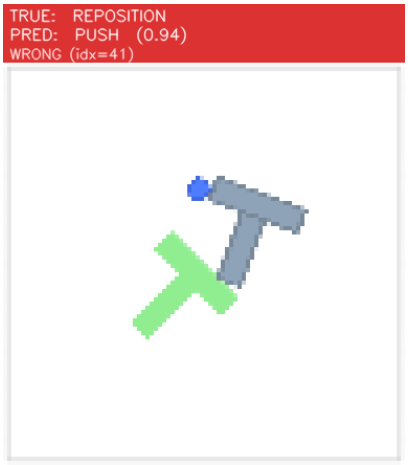

Figure 7: π_{high} misclassification failure. The expert would select REPOSITION at this state (block partially rotated, end-effector at top contact), but π_{high} predicts PUSH with 0.94 confidence. At inference, this locks π_{low} into the PUSH option while π_{low} predicts a near-zero remaining duration, causing the high and low policies to disagree and the robot to stall.

Rollout trajectory visualizations comparing SODA-D and DP and per-episode failure mode breakdowns across seeds are left as future work pending additional evaluation infrastructure.

6 Discussion

Receptive field and option structure. The kernel size study reveals that option-conditioned diffusion requires a wider temporal receptive field than flat Diffusion Policy. Push-T option segments average $\sim$17 steps, and the U-Net must capture dependencies across the full suffix to generate coherent stretched action targets. The monotonic improvement from k=5 to k=9 reflects this: the default k=5 used by the Columbia DP baseline is simply too narrow for the longer planning horizons that options impose on π_{low}. However, it remains unclear whether this benefit is specific to SODA or whether a flat Diffusion Policy trained with k=7 or k=9 would show similar gains. A direct comparison training base DP at matched kernel sizes is a necessary next step to determine whether the performance improvement from larger k is unique to the option-conditioned setting (where longer action chunks benefit more from wider context) or is a general property of increasing the receptive field independent of the hierarchical structure.

Duration termination as a strong baseline. The termination study is the most instructive negative result of this work. SODA-B is theoretically the more capable execution strategy it can exit to π_{high} or π_{low} on demand at the cost of a single MLP forward pass per step. Yet every variant we evaluated trails the duration baseline by at least 8 percentage points. The most likely explanation is that Push-T's option segments are sufficiently regular in duration that the predicted scalar is a reliable proxy for completion, leaving little room for a learned head to add value. Whether this holds on tasks with more variable or visually ambiguous option boundaries remains an open question, and is a key motivation for extending SODA to more complex manipulation settings.

Noise robustness. Perhaps the most striking result is the robustness gap under temporally-correlated noise. DP degrades by 17.2 percentage points at η=1, while SODA-D degrades by only 0.3 points effectively flat. Both policies are evaluated at T_a=1, so both replan at every step; the advantage for SODA-D is not simply due to more frequent replanning, but to the quality of each replan. Because π_{low} plans over a horizon scoped to the current semantic option rather than a global fixed window, each new plan is anchored to a meaningful subtask objective and less susceptible to noise-induced drift. A consequence of T_a=1 is that SODA-D also lacks chunk-to-chunk consistency at η=0 BID applied on top of SODA-D could recover consistency and potentially improve clean performance further, at the cost of N diffusion passes per step.

Disentangling hierarchy from receptive field. The headline comparison SODA-D k=9 vs. DP k=5 conflates two factors: the hierarchical decomposition and the wider receptive field. The matched-kernel comparison (k=5 vs. k=5, +4.3 pp) isolates hierarchy, though with limited power at 50 episodes per single training run. A DP k=9 control run would cleanly attribute the remaining gap, and is planned as follow-up work.

Limitations. The current results are limited to a single 2D task with VLM-generated option labels. The generalization of SODA's gains to higher-dimensional manipulation, contact-rich tasks with less regular option structure, or automatically discovered options remains to be demonstrated. The termination study also did not fully explore the design space several combinations (bottleneck features with escape relabeling, standalone ResNet-based β decoupled from π_{low}) were not evaluated due to compute constraints. The main comparison (Table 2) reports a single training run per method; a more rigorous evaluation would train each method from three independent random seeds and evaluate each checkpoint over 50 episodes, using the resulting distribution of means for statistical comparison and variance estimation. We were limited by the compute cost of training SODA from scratch, which made multi-seed evaluation impractical within the project scope. Future work will extend SODA to 6-DOF manipulation with VLM-supervised options (Square, E2) and unsupervised option discovery via LOVE (E3/E4).

7 Conclusion

We presented SODA, a hierarchical imitation learning framework that decomposes manipulation tasks into discrete semantic options and plans variable-horizon action chunks matched to each option's duration. On Push-T, SODA-D with k=9 achieves 98.0% mean max block-target overlap, outperforming the Columbia Diffusion Policy baseline (89.0%) with substantially reduced variance. Hierarchy contributes even at matched kernel size (k=5: +4.3 pp), confirming that the benefit is not solely due to a wider receptive field.

The most instructive finding is a negative result: across eight variants of the learned termination head β, none approached the performance of the duration channel baseline, which outperformed every learned variant by at least 8 percentage points. This establishes duration termination as a surprisingly strong mechanism for option switching and raises the question of whether more complex tasks with less regular option structure will provide the signal that β needs to add value.

SODA's modular architecture with option discovery, low-level diffusion, and termination as independently configurable components makes it a natural platform for systematic study. Future work will extend evaluation to 6-DOF manipulation with VLM-supervised options (Square, E2) and unsupervised option discovery via LOVE (E3/E4), where the regularity assumptions that favor duration termination may not hold.

8 Team Contributions

Umar Padela implemented the full SODA architecture (π_{low}, π_{high}, termination head, hierarchical controller), training and evaluation infrastructure (Modal GPU pipeline, W&B logging, zarr dataset pipeline), and conducted all Push-T experiments: kernel size study, termination study, receding horizon study, SODA vs. DP comparison, and noise robustness study. Umar also built the VLM option labeling pipeline for Push-T.

Neetish Sharma worked on unsupervised option discovery, integrating the LOVE framework for automated segmentation of manipulation demonstrations.

Changes from Proposal The proposal planned results across multiple simulation environments. The final report narrows scope to Push-T, where the depth of the kernel, termination, receding horizon, and noise studies provides thorough evaluation of the SODA framework. Results on additional environments and option discovery methods were not completed in time for the final report. The termination study expanded significantly beyond the original plan into a systematic eight-variant study; the negative result that duration termination outperforms all learned β variants is itself a contribution.

References

Jacob Andreas, Dan Klein, and Sergey Levine. 2017. Modular Multitask Reinforcement Learning with Policy Sketches. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 166–175.

Akhil Bagaria and George Konidaris. 2020. Option Discovery using Deep Skill Chaining. In *International Conference on Learning Representations (ICLR)*. `https://openreview.net/forum?id=B1gqipNYwH`

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2023. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*. `https://arxiv.org/abs/2303.04137`

Yiding Jiang, Evan Zheran Guo, Benjamin Eysenbach, Chelsea Finn, and J. Zico Kolter. 2022. Learning Options via Compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35.

Thomas Kipf, Yujia Li, Hanjun Dai, Vinicius Zambaldi, Alvaro Sanchez-Gonzalez, Edward Grefenstette, Pushmeet Kohli, and Peter Battaglia. 2019. CompILE: Compositional Imitation Learning and Execution. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*. PMLR, 3418–3428.

Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Max Du, and Chelsea Finn. 2024. Bidirectional Decoding: Improving Action Chunking via Closed-Loop Resampling. arXiv:2408.17355 [cs.RO]

Samuel Pfrommer, Dongyoon Shi, Tobias Kolb, Hassan Shafiee, and J. Andrew Bagnell. 2024. Flow Matching with General Discrete Curved Spaces. See also: variable-horizon diffusion with horizon channel for min-time RL. arXiv:2402.07307 [cs.LG]

Kyriacos Shiarlis, Markus Wulfmeier, Sasha Salter, Shimon Whiteson, and Ingmar Posner. 2018. TACO: Learning Task Decomposition via Temporal Alignment for Control. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 4654–4663.

Richard S. Sutton, Doina Precup, and Satinder Singh. 1999. Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence* 112, 1–2 (1999), 181–211.

Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Proceedings of Robotics: Science and Systems (RSS)*. `https://arxiv.org/abs/2304.13705`